A CAMADA DE HARDWARE

4 - Portas e Circuitos

Computadores são dispositivos eletrônicos; os elementos de hardware mais fundamentais de um computador controlam o fluxo de eletricidade. Em um sentido muito primitivo, usamos a tecnologia para aproveitar o poder de um raio, dobrando-o à nossa vontade para que possamos realizar cálculos e tomar decisões. Este capítulo caminha na tênue linha entre a ciência da computação e a engenharia elétrica, examinando os elementos de hardware mais básicos de um computador.

No Capítulo 2, examinamos os sistemas numéricos em geral e o sistema numérico binário em particular. Como vimos no Capítulo 3, o sistema numérico binário é de especial interesse porque é usado para representar dados em um computador. Neste capítulo, exploramos como os computadores usam sinais elétricos para representar e manipular esses valores binários.

OBJETIVOS

Após estudar este capítulo, você deverá ser capaz de:

- identificar as portas básicas e descrever o comportamento de cada uma.
- descrever como as portas são implementadas usando transistores.
- combinar portas básicas em circuitos.
- descrever o comportamento de uma porta ou circuito usando expressões booleanas, tabelas verdade e diagramas lógicos.
- comparar e contrastar um meio somador e um somador completo.
- descrever como um multiplexador funciona.
- explicar como funciona uma trava S-R.
- descrever as características das quatro gerações de circuitos integrados.

4.1 Computadores e Eletricidade

Qualquer sinal eletrônico possui um nível de tensão. Como mencionamos no capítulo anterior, distinguimos os dois valores de interesse (binário 0 e 1) pelo nível de tensão de um sinal. Em geral, um nível de tensão na faixa de 0 a 2 volts é considerado "baixo" e é interpretado como um binário 0. Um sinal na faixa de 3 a 5 volts é considerado "alto" e é interpretado como um binário 1. Os sinais em um computador são limitados a estar dentro de uma ou outra faixa.

Uma porta é um dispositivo que realiza uma operação básica em sinais elétricos. Ela aceita um ou mais sinais de entrada e produz um único sinal de saída. Existem vários tipos de portas; examinaremos os seis tipos mais fundamentais neste capítulo. Cada tipo de porta executa uma função lógica específica.

* Porta: Um dispositivo que realiza uma operação básica em sinais elétricos, aceitando um ou mais sinais de entrada e produzindo um único sinal de saída.

Portas são combinadas em circuitos para realizar tarefas mais complexas. Por exemplo, circuitos podem ser projetados para realizar operações aritméticas e armazenar valores. Em um circuito, o valor de saída de uma porta frequentemente serve como valor de entrada para uma ou mais outras portas. O fluxo de eletricidade através de um circuito é controlado pela lógica cuidadosamente projetada das portas interativas.

* Circuito: Uma combinação de portas interativas projetadas para realizar uma função lógica específica.

Três métodos de notação diferentes, mas igualmente poderosos, são usados para descrever o comportamento de portas e circuitos:

- Expressões booleanas
- Diagramas lógicos
- Tabelas verdade

Examinamos todos os três tipos de representação durante nossa discussão sobre portas e circuitos.

O matemático inglês George Boole inventou uma forma de álgebra na qual variáveis e funções assumem apenas um de dois valores possíveis (0 e 1). Essa álgebra é apropriadamente chamada de álgebra booleana. Expressões nessa notação algébrica são uma maneira elegante e poderosa de demonstrar a atividade de circuitos elétricos. Operações e propriedades específicas da álgebra booleana nos permitem definir e manipular a lógica de circuitos usando uma notação matemática. As expressões booleanas serão abordadas novamente em nossas discussões sobre a camada de programação nos Capítulos 6 a 9.

Um diagrama lógico é uma representação gráfica de um circuito. Cada tipo de porta é representado por um símbolo gráfico específico. Conectando esses símbolos de várias maneiras, podemos representar visualmente a lógica de um circuito inteiro.

Uma tabela-verdade define a função de uma porta listando todas as combinações de entrada possíveis que a porta pode encontrar, juntamente com a saída correspondente. Podemos projetar tabelas-verdade mais complexas, com linhas e colunas suficientes para mostrar o desempenho de circuitos inteiros para qualquer conjunto de valores de entrada.

* Álgebra booleana: Uma notação matemática para expressar funções lógicas de dois valores.

* Diagrama lógico: Uma representação gráfica de um circuito; cada tipo de porta possui seu próprio símbolo.

* Tabela verdade Uma tabela que mostra todos os valores de entrada possíveis e os valores de saída associados.

George Boole[1]

A álgebra booleana recebeu o nome de seu inventor, o matemático inglês George Boole, nascido em 1815. Seu pai, um comerciante, começou a lhe ensinar matemática ainda jovem. Mas Boole inicialmente se interessava mais por literatura clássica, línguas e religião — interesses que manteve por toda a vida. Aos 20 anos, já havia aprendido francês, alemão e italiano sozinho. Era versado nos escritos de Aristóteles, Espinosa, Cícero e Dante, e escreveu vários artigos filosóficos.

Aos 16 anos, assumiu o cargo de assistente de ensino em uma escola particular para ajudar a sustentar sua família. Seu trabalho lá, somado a um segundo emprego como professor, lhe deixavam pouco tempo para estudar. Alguns anos depois, abriu uma escola e começou a aprender matemática avançada por conta própria. Apesar da falta de treinamento formal, seu primeiro artigo acadêmico foi publicado no Cambridge Mathematical Journal quando ele tinha apenas 24 anos. Em 1849, foi nomeado professor de matemática no Queen's College, em Cork, Irlanda. Tornou-se catedrático de matemática e passou o resto de sua carreira lá. Boole publicou mais de 50 artigos e diversas obras importantes antes de falecer em 1864, no auge de sua carreira.

A obra "A Análise Matemática da Lógica", de Boole, foi publicada em 1847. Ela viria a formar a base para o desenvolvimento dos computadores digitais. No livro, Boole expôs os axiomas formais da lógica (muito semelhantes aos axiomas da geometria) sobre os quais o campo da lógica simbólica se baseia. Boole baseou-se nos símbolos e operações da álgebra para criar seu sistema de lógica. Ele associou o valor 1 ao conjunto universal (o conjunto que representa tudo no universo) e o valor 0 ao conjunto vazio, e restringiu seu sistema a essas grandezas. Em seguida, definiu operações análogas à subtração, adição e multiplicação. Em 1854, Boole publicou "Uma Investigação das Leis do Pensamento; sobre as Quais São Fundadas as Teorias Matemáticas da Lógica e das Probabilidades". Este livro descreveu teoremas baseados em seus axiomas da lógica e estendeu a álgebra para mostrar como as probabilidades poderiam ser computadas em um sistema lógico. Cinco anos depois, Boole publicou o Tratado sobre Equações Diferenciais, seguido pelo Tratado sobre o Cálculo das Diferenças Finitas. Este último é um dos pilares da análise numérica, que trata da precisão dos cálculos.

Boole recebeu pouco reconhecimento e poucas homenagens por seu trabalho. Dada a importância da álgebra booleana na tecnologia moderna, é difícil acreditar que seu sistema de lógica não tenha sido levado a sério até o início do século XX. George Boole foi realmente um dos fundadores da ciência da computação.

? O que é nanociência?

Nanociência e nanotecnologia são o estudo e a aplicação de coisas extremamente pequenas:

- Há 25.400.000 nanômetros em uma polegada.
- Uma folha de jornal tem cerca de 100.000 nanômetros de espessura.
- Se uma bola de gude fosse um nanômetro, um metro seria o tamanho da Terra.[2]

4.2 Portas

As portas em um computador são às vezes chamadas de portas lógicas porque cada uma executa apenas uma função lógica. Ou seja, cada porta aceita um ou mais valores de entrada e produz um único valor de saída. Como estamos lidando com informações binárias, cada valor de entrada e saída é 0, correspondendo a um sinal de baixa tensão, ou 1, correspondendo a um sinal de alta tensão. O tipo de porta e os valores de entrada determinam o valor de saída.

Vamos examinar o processamento dos seis tipos de portas a seguir. Em seguida, mostraremos como elas podem ser combinadas em circuitos para realizar operações aritméticas.

- NOT (NÃO)
- AND (E)
- OR (OU)
- XOT (OU exclusivo)
- NAND
- NOR

Neste livro, colorimos os símbolos do diagrama lógico para cada porta para ajudar você a controlar os vários tipos. Quando examinamos circuitos completos com muitas portas, as cores ajudarão você a distinguí-las. No mundo real, no entanto, os diagramas lógicos são tipicamente preto e branco, e as portas são distinguidas apenas por sua forma.

Porta NOT

Uma porta NOT aceita um valor de entrada e produz um valor de saída. A FIGURA 4.1 mostra uma porta NOT representada de três maneiras: como uma expressão booleana, como seu símbolo de diagrama lógico e por meio de uma tabela-verdade. Em cada representação, a variável A representa o sinal de entrada, que é 0 ou 1. A variável X representa o sinal de saída, cujo valor (0 ou 1) é determinado pelo valor de A.

Por definição, se o valor de entrada de uma porta NOT for 0, o valor de saída será 1; se o valor de entrada for 1, a saída será 0. Uma porta NOT às vezes é chamada de inversora porque inverte o valor de entrada.

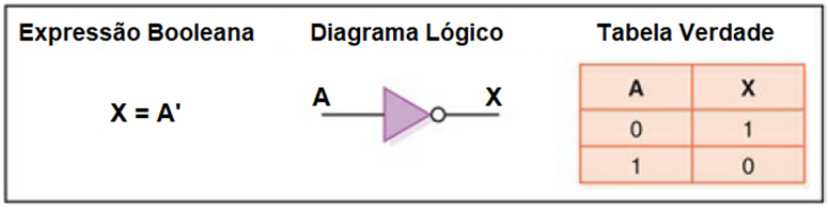


FIGURA 4.1 Representações de uma porta NOT.

Em expressões booleanas, a operação NOT é representada pela marcação de um apóstrofo (') após o valor ser negado. Às vezes, essa operação é mostrada com uma barra horizontal sobre o valor a ser negado. Na expressão booleana da Figura 4.1, X recebe o valor determinado pela aplicação da operação NOT ao valor de entrada A. Em tal instrução de atribuição, a variável à esquerda do sinal de igual assume o valor da expressão à direita. As instruções de atribuição são discutidas mais detalhadamente no Capítulo 6. O símbolo do diagrama lógico para uma porta NOT é um triângulo com um pequeno círculo (chamado de bolha de inversão) na extremidade. A entrada e a saída são mostradas como linhas fluindo para dentro e para fora da porta. Às vezes, essas linhas são rotuladas, mas nem sempre.

A tabela verdade na Figura 4.1 mostra todos os valores de entrada possíveis para uma porta NOT, bem como os valores de saída correspondentes. Como há apenas um sinal de entrada para uma porta NOT, e esse sinal pode ser apenas 0 ou 1, há apenas duas possibilidades para a coluna rotulada A na tabela verdade. A coluna rotulada X mostra a saída da porta, que é o inverso da entrada. Observe que, das três representações, apenas a tabela verdade define o comportamento da porta para todas as situações.

Essas três notações são apenas maneiras diferentes de representar a mesma coisa. Por exemplo, o resultado da expressão booleana

0′ é sempre 1,

e o resultado da expressão booleana

1′ é sempre 0.

Esse comportamento é consistente com os valores mostrados na tabela verdade.

Porta AND

A FIGURA 4.2 descreve uma porta AND. Ao contrário de uma porta NOT, que aceita um sinal de entrada, uma porta AND aceita dois sinais de entrada. Os valores de ambos os sinais de entrada determinam qual será o sinal de saída. Se os dois valores de entrada para uma porta AND forem ambos 1, a saída será 1; caso contrário, a saída será 0.

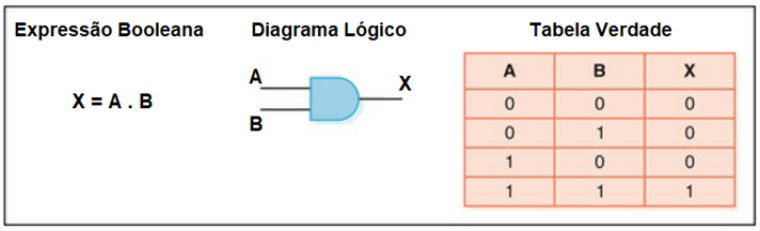


FIGURA 4.2 Representações de uma porta AND.

A operação AND na álgebra booleana é expressa usando um único ponto (.). Às vezes, um asterisco (*) é usado para representar esse operador. Frequentemente, o próprio operador é suprimido. Por exemplo, A·B é frequentemente escrito como AB.

Como há duas entradas e dois valores possíveis para cada entrada, quatro combinações possíveis de 1 e 0 podem ser fornecidas como entrada para uma porta AND. Portanto, quatro situações podem ocorrer quando o operador AND é usado em uma expressão booleana:

0 · 0 é igual a 0; 0 · 1 é igual a 0; 1 · 0 é igual a 0 e 1 · 1 é igual a 1.

Da mesma forma, a tabela-verdade (FIGURA 4.2) que mostra o comportamento da porta AND tem quatro linhas, mostrando todas as quatro combinações possíveis de entrada. A coluna de saída da tabela-verdade é consistente com os resultados dessas expressões booleanas.

Porta OR

A FIGURA 4.3 mostra uma porta OR. Assim como a porta AND, a porta OR possui duas entradas. Se ambos os valores de entrada forem 0, o valor de saída será 0; caso contrário, a saída será 1.

A operação OR da álgebra booleana é expressa usando um sinal de mais (+). A porta OR possui duas entradas, cada uma das quais pode ser um de dois valores. Assim, como em uma porta AND, há quatro combinações de entrada e, portanto, quatro linhas na tabela verdade.

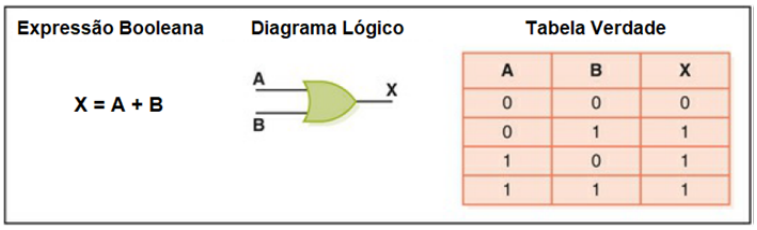


FIGURA 4.3 Representações de uma porta OU.

Porta XOR

A porta XOR, ou porta exclusive-OR (OU exclusiva), é mostrada na FIGURA 4.4. Para duas entradas, uma porta XOR produz um 0 se ambas entradas forem iguais e um 1 caso contrário. Observe a diferença entre a porta XOR e a porta OU; elas diferem em apenas uma situação de entrada. Quando ambos os sinais de entrada são 1, a porta OU produz um 1 e o XOR produz um 0.

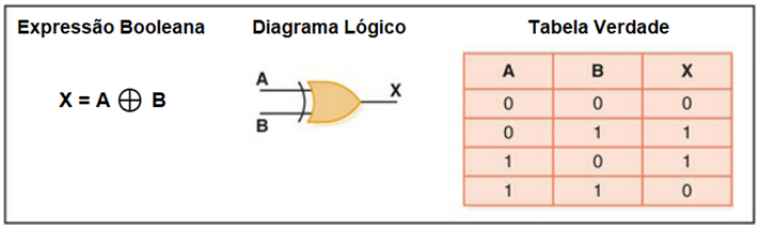


FIGURA 4.4 Representações de uma porta XOR.

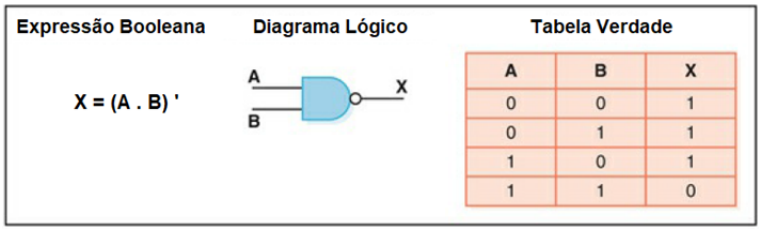
Às vezes, a porta OR regular é chamada de inclusive-OR (OU inclusivo), porque produz um 1 se uma ou ambas as suas entradas forem 1. O XOR produz um 1 somente se suas entradas forem misturadas (uma 1 e uma 0). Pense na porta XOR como se dissesse: "Quando digo ou, quero dizer um ou outro, não ambos".

O símbolo de álgebra booleana $\oplus$ às vezes é usado para expressar a operação XOR. Além disso, a operação XOR pode ser expressa usando os outros operadores; deixamos isso como um exercício.

Observe que o símbolo do diagrama lógico para a porta XOR é igual ao símbolo para uma porta OR, exceto por ter uma linha curva extra conectando seus sinais de entrada.

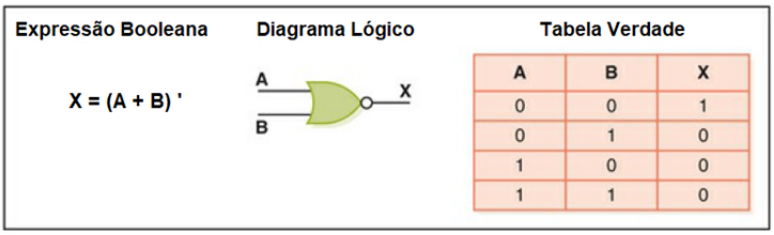
Portas NAND e NOR

A porta NAND é mostrada na FIGURA 4.5 e a porta NOR é mostrada na FIGURA 4.6. Cada uma aceita dois valores de entrada. As portas NAND e NOR são essencialmente os opostos das portas AND e OR, respectivamente. Ou seja, a saída de uma porta NAND é a mesma que se você pegasse a saída de uma porta AND e a colocasse em um inversor (uma porta NOT).



A	B	X
0	0	1
0	1	1
1	0	1
1	1	0

FIGURA 4.5 Representações de uma porta NAND



A	B	X
0	0	1
0	1	0
1	0	0
1	1	0

FIGURA 4.6 Representações de uma porta NOR

Nenhum símbolo específico é usado para expressar as operações NAND e NOR na álgebra booleana. Em vez disso, nos baseamos em suas definições para expressar os conceitos. Ou seja, a expressão da álgebra booleana para NAND é a negação de uma operação AND. Da mesma forma, a expressão da álgebra booleana para NOR é a negação de uma operação OR.

Os símbolos do diagrama lógico para as portas NAND e NOR são os mesmos para as portas AND e OR, exceto que os símbolos NAND e NOR usam uma bolha de inversão (para indicar a negação). Compare as colunas de saída das tabelas verdade para as portas AND e NAND. Elas são opostas, quando você as observa linha por linha. O mesmo vale para as portas OR e NOR.

? A décima vertente

O currículo de computação de 1991, um relatório de uma força-tarefa conjunta da Association for Computing Machinery – ACM (ou Associação para Máquinas de Computação) e do Institute of Electrical and Electronics Engineers – IEEE (ou Instituto de Engenheiros Eletricistas e Eletrônicos), apresentou recomendações para o design de currículos de bacharelado em ciência da computação. Embora o relatório tivesse uma seção intitulada "Contexto Social e Profissional", que afirmava que os alunos precisam "compreender as questões culturais, sociais, legais e éticas básicas inerentes à computação", o estudo da ética não era uma das nove áreas temáticas, ou vertentes. Em 2001, questões sociais e profissionais foram incluídas como uma área temática no ensino de computação e chamadas de décima vertente.

Revisão do Processamento de Portas

Analisamos seis tipos específicos de portas. Pode parecer uma tarefa difícil mantê-las em ordem e lembrar como todas funcionam – mas isso provavelmente depende de como você pensa a respeito. Definitivamente, não recomendamos que você tente memorizar tabelas-verdade. O processamento dessas portas pode ser

descrito brevemente em termos gerais. Se você pensar nelas dessa forma, poderá produzir a tabela-verdade apropriada sempre que precisar.

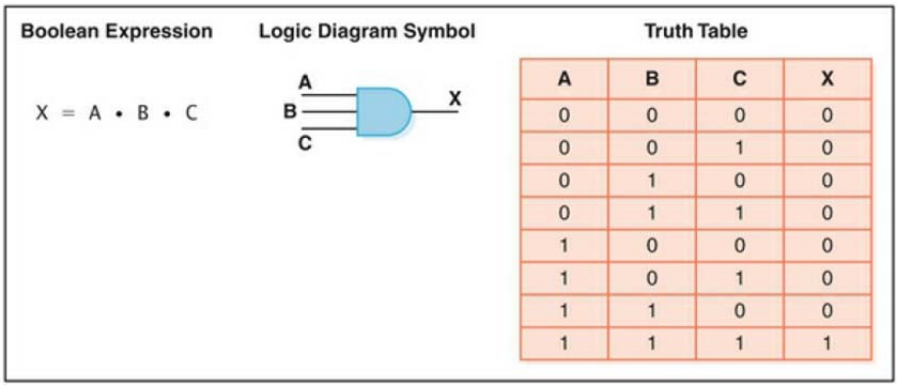
Vamos revisar o processamento de cada porta. Algumas dessas descrições são apresentadas em termos de quais valores de entrada fazem com que a porta produza 1 como saída; em qualquer outro caso, a porta produz 0.

- Uma porta NOT inverte seu único valor de entrada.
- Uma porta AND produz 1 se ambos os valores de entrada forem 1.
- Uma porta OR produz 1 se pelos menos uma de seus valores de entrada, for 1.
- Uma porta XOR produz 1 se um ou outro (mas não ambos) os valores de entrada forem 1.
- Uma porta NAND produz os resultados invertidos de uma porta AND.
- Uma porta NOR produz os resultados invertidos de uma porta OR.

Com essas regras gerais de processamento em mente, tudo o que resta é lembrar os operadores booleanos e os símbolos do diagrama lógico. Lembre-se de que vários símbolos do diagrama lógico são variações de outros símbolos do diagrama lógico. Além disso, lembre-se de que a coloração das portas neste livro tem como objetivo ajudá-lo a acompanhar os vários tipos de portas; tradicionalmente, elas são simplesmente diagramas em preto e branco.

Portas com Mais Entradas

Portas podem ser projetadas para aceitar três ou mais valores de entrada. Uma porta AND de três entradas, por exemplo, produz uma saída 1 somente se todos os valores de entrada forem 1. Uma porta OR de três entradas produz uma saída 1 se qualquer valor de entrada for 1. Essas definições são consistentes com as versões de duas entradas dessas portas. A FIGURA 4.7 mostra uma porta AND com três sinais de entrada.



A	B	C	X
0	0	0	0
0	0	1	0
0	1	0	0
0	1	1	0
1	0	0	0
1	0	1	0
1	1	0	0
1	1	1	1

FIGURA 4.7 Representações de uma porta AND de três entradas.

Existem 2^3, ou 8, combinações de entrada possíveis para uma porta com três entradas. Lembre-se do Capítulo 3 que existem 2^n combinações de 1 e 0 para n valores de entrada distintos. Este número determina quantas linhas aparecem na tabela verdade.

Para o símbolo do diagrama lógico, simplesmente adicionamos um terceiro sinal de entrada ao símbolo original. Para uma expressão booleana, repetimos a operação AND para representar o terceiro valor.

4.3 Construindo Portas

Antes de examinarmos como as portas são conectadas para formar circuitos, vamos examinar, em um nível ainda mais básico, como uma porta é construída para controlar o fluxo de eletricidade.

Transistores

Uma porta usa um ou mais transistores para estabelecer como os valores de entrada se relacionam com o valor de saída. Um transistor é um dispositivo que atua, dependendo do nível de tensão do sinal de entrada, como um fio que conduz eletricidade ou como um resistor que bloqueia o fluxo de eletricidade. Um

transistor não possui partes móveis, mas atua como uma chave. Ele é feito de um material semicondutor que não é um condutor de eletricidade particularmente bom (ao contrário do cobre) nem um isolante particularmente bom (ao contrário da borracha). Normalmente, o silício é usado para criar transistores.

* Transistor: Dispositivo que atua como um fio ou um resistor, dependendo do nível de tensão de um sinal de entrada.

* Semicondutor: Material como o silício, que não é um bom condutor nem um bom isolante.

No Capítulo 1, mencionamos que a invenção dos transistores, ocorrida em 1947 nos Bell Labs, mudou a face da tecnologia, inaugurando a segunda geração de hardware de computador. Antes do advento dos transistores, os circuitos digitais usavam válvulas de vácuo, que dissipavam muito calor e frequentemente falhavam, exigindo substituição. Os transistores são muito menores que as válvulas de vácuo e requerem menos energia para operar. Eles podem alternar estados em poucos nanossegundos. A computação, como a conhecemos hoje, deve-se em grande parte à invenção do transistor.

Antes de abordar os detalhes dos transistores, vamos discutir alguns princípios básicos da eletricidade. Um sinal elétrico tem uma fonte, como uma bateria ou uma tomada na parede. Se o sinal elétrico estiver aterrado, ele poderá fluir por uma rota alternativa até o solo (literalmente), onde não causará danos. Um sinal elétrico aterrado é puxado para baixo, ou reduzido, a 0 volts.

Um transistor possui três terminais: uma fonte, uma base e um emissor. O emissor é tipicamente conectado a um fio terra, como mostrado na FIGURA 4.8 (ao lado). Para computadores, a fonte produz um valor de alta tensão, aproximadamente 5 volts. O valor da base regula uma porta que determina se a conexão entre a fonte e o terra é feita. Se o sinal da fonte estiver aterrado, ele é puxado para baixo a 0 volts. Se a base não aterrar o sinal da fonte ele permanece alto.

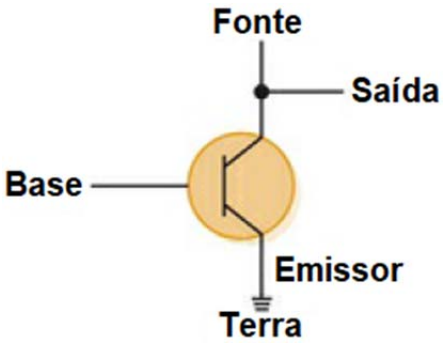


FIGURA 4.8 As conexões de um transistor.

Uma linha de saída é geralmente conectada à linha de fonte. Se o sinal da fonte for puxado para o terra pelo transistor, o sinal de saída será baixo, representando um binário 0. Se o sinal da fonte permanecer alto, o sinal de saída também permanecerá, representando um binário 1.

O transistor está ligado, produzindo um sinal de saída alto, ou desligado, produzindo um sinal de saída baixo. Essa saída é determinada pelo sinal elétrico da base. Se o sinal da base estiver alto (próximo a uma tensão de +5V), o sinal da fonte será aterrado, o que desliga o transistor. Se o sinal da base estiver baixo (próximo a uma tensão de 0V), o sinal da fonte permanecerá alto e o transistor estará ligado.

Agora, vamos ver como um transistor é usado para criar vários tipos de portas. Acontece que, devido à forma como um transistor funciona, as portas mais fáceis de criar são as portas NOT, NAND e NOR. A FIGURA 4.9 mostra como essas portas podem ser construídas usando transistores.

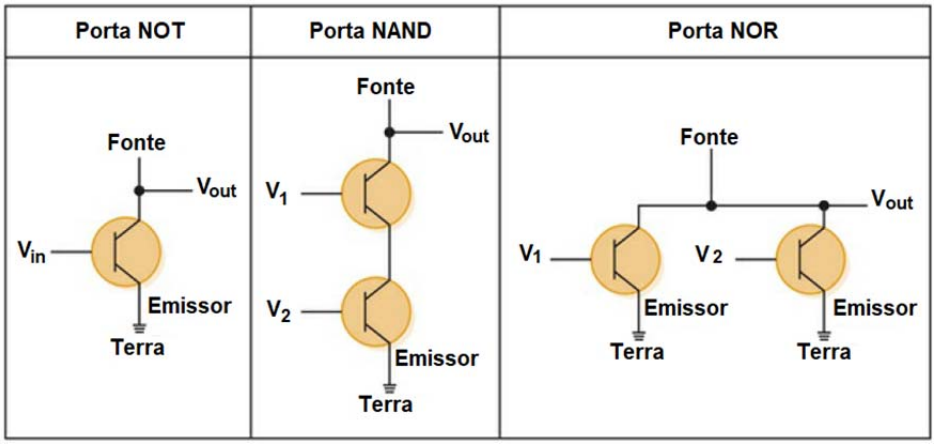


FIGURA 4.9 Construindo portas usando transistores

O diagrama da porta NOT é essencialmente o mesmo do nosso diagrama de transistor original. É necessário apenas um transistor para criar uma porta NOT. O sinal V_{in} representa o sinal de entrada para a porta NOT. Se for alto, a fonte está aterrada e o sinal de saída V_{out} é baixo. Se V_{in} for baixo, a fonte não está aterrada e V_{out} é alto. Portanto, o sinal de entrada é invertido, que é exatamente o que uma porta NOT faz.

A porta NAND requer dois transistores. Os sinais de entrada V_1 e V_2 representam a entrada para a porta NAND. Se ambos os sinais de entrada estiverem em nível alto, a fonte está aterrada e a saída V_{out} está em nível baixo. Se qualquer um dos sinais de entrada estiver em nível baixo, no entanto, um ou outro transistor impede que o sinal da fonte seja aterrado e a saída está em nível alto. Portanto, se V_1 ou V_2, ou ambos, carregam um sinal baixo (binário 0), a saída é 1. Isso é consistente com o processamento de uma porta NAND.

A construção de uma porta NOR também requer dois transistores. Mais uma vez, V_1 e V_2 representam a entrada para a porta. Desta vez, no entanto, os transistores não estão conectados em série. A fonte se conecta a cada transistor separadamente. Se qualquer um dos transistores permitir que o sinal da fonte seja aterrado, a saída é 0. Portanto, a saída é alta (binário 1) apenas quando V_1 e V_2 estão em nível baixo (binário 0), que é o que queremos para uma porta NOR.

Uma porta AND produz uma saída exatamente oposta à saída de uma porta NAND. Portanto, para construir uma porta AND, simplesmente passamos a saída de uma porta NAND por um inversor (uma porta NOT). É por isso que as portas AND são mais complicadas de construir do que as portas NAND: elas requerem três transistores, dois para a NAND e um para a NOT. O mesmo raciocínio pode ser aplicado para entender a relação entre as portas NOR e OR.

4.4 Circuitos

Agora que sabemos como as portas individuais funcionam e como são construídas, vamos examinar como combinamos portas para formar circuitos. Os circuitos podem ser classificados em duas categorias gerais. Em um circuito combinacional, os valores de entrada determinam explicitamente a saída. Em um circuito sequencial, a saída é uma função dos valores de entrada, bem como do estado atual do circuito. Assim, circuitos sequenciais geralmente envolvem o armazenamento de informações. A maioria dos circuitos que examinamos neste capítulo são circuitos combinacionais, embora mencionemos brevemente os circuitos de memória sequencial.

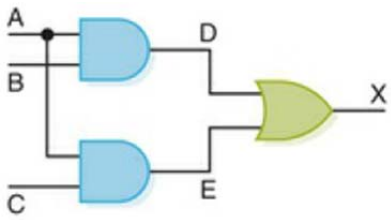
* Circuito combinacional: Um circuito cuja saída é determinada exclusivamente por seus valores de entrada.

* Circuito sequencial: Um circuito cuja saída é uma função de seus valores de entrada e do estado atual do circuito.

Assim como com as portas, podemos descrever as operações de circuitos inteiros usando três notações: expressões booleanas, diagramas lógicos e tabelas verdade. Essas notações são técnicas de representação diferentes, mas igualmente poderosas.

Circuitos Combinacionais

Portas são combinadas em circuitos usando a saída de uma porta como entrada para outra porta. Por exemplo, considere o seguinte diagrama lógico de um circuito:



A saída das duas portas AND é usada como entrada para a porta OR. O valor de entrada A é usado como entrada para ambas as portas AND. O ponto indica que duas linhas estão conectadas. Se a interseção de duas linhas cruzadas não tiver um ponto, você deve pensar em uma como se estivesse "saltando sobre" a outra sem afetar uma à outra.

O que significa este diagrama lógico? Bem, vamos trabalhar de trás para frente para ver o que é necessário para obter um resultado específico. Para que a saída final X seja 1, D deve ser 1 ou E deve ser 1, ou ambas. Para que D seja 1, A e B devem ser 1. Para que E seja 1, A e C devem ser 1. Tanto E quanto D podem ser 1, mas isso não é necessário. Examine este diagrama de circuito cuidadosamente; certifique-se de que este raciocínio seja consistente com sua compreensão dos tipos de portas utilizadas.

Agora, vamos representar o processamento de todo este circuito usando uma tabela-verdade (vide tabela ao lado).

A	B	C	D	E	X
0	0	0	0	0	0
0	0	1	0	0	0
0	1	0	0	0	0
0	1	1	0	0	0
1	0	0	0	0	0
1	0	1	0	1	1
1	1	0	1	0	1
1	1	1	1	1	1

Como este circuito possui três entradas, são necessárias oito linhas para descrever todas as combinações possíveis de entradas. As colunas intermediárias mostram os valores intermediários (D e E) no circuito.

Por fim, vamos expressar este mesmo circuito usando álgebra booleana. Um circuito é uma coleção de portas que interagem, portanto, uma expressão booleana para representar um circuito é uma combinação das operações booleanas apropriadas. Precisamos apenas juntar as operações na forma correta para criar uma expressão de álgebra booleana válida. Neste circuito, existem duas expressões AND. A saída de cada operação AND é a entrada para a operação OR. Portanto, este circuito é representado pela seguinte expressão booleana (na qual o operador AND é assumido):

(AB + AC)

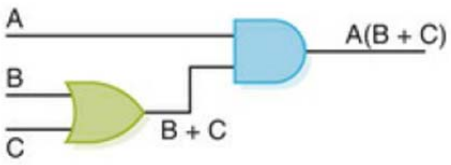
Quando escrevemos tabelas-verdade, geralmente é melhor rotular as colunas usando esses tipos de expressões booleanas em vez de variáveis arbitrárias como D, E e X. Isso deixa claro o que cada coluna representa. Na verdade, podemos usar expressões booleanas para rotular nossos diagramas lógicos também, eliminando completamente a necessidade de variáveis intermediárias.

Agora, vamos por outro caminho: vamos pegar uma expressão booleana e desenhar o diagrama lógico e a tabela-verdade correspondentes. Considere a seguinte expressão booleana:

A(B + C)

Nesta expressão, a operação OR é aplicada aos valores de entrada B e C. O resultado dessa operação é usado como entrada, juntamente com A, para uma operação AND, produzindo o resultado final.

O diagrama de circuito correspondente é representado na figura ao lado.



Mais uma vez, completamos nossa série de representações expressando este circuito como uma tabela-verdade. Como no exemplo anterior, há três valores de entrada, portanto, há oito linhas na tabela-verdade:

Escolha uma linha desta tabela-verdade e siga a lógica do diagrama de circuito para garantir que os resultados finais sejam consistentes. Experimente com algumas linhas para se familiarizar com o processo de traçar a lógica de um circuito.

Agora compare a coluna do resultado final nesta tabela-verdade com a tabela-verdade do exemplo anterior. Elas são idênticas. Acabamos de demonstrar a equivalência de circuitos. Ou seja, ambos os circuitos produzem exatamente a mesma saída para cada combinação de valor de entrada.

A	B	C	B+C	A(B+C)
0	0	0	0	0
0	0	1	1	0
0	1	0	1	0
0	1	1	1	0
1	0	0	0	0
1	0	1	1	1
1	1	0	1	1
1	1	1	1	1

* Equivalência de circuitos: A mesma saída para cada combinação correspondente de valor de entrada para dois circuitos.

De fato, esta situação demonstra uma propriedade importante da álgebra booleana chamada **lei distributiva**:

A(B + C) = AB + AC

Essa é a beleza da álgebra booleana: ela nos permite aplicar princípios matemáticos comprováveis para projetar circuitos lógicos. O gráfico a seguir mostra algumas propriedades da álgebra booleana:

Propriedade	AND	OR
Comutativa	AB = BA	A+B = B+A
Associativa	(AB)C = A(BC)	(A+B)+C = A+(B+C)
Distributiva	A(B+C) = (AB)+(AC)	A+(BC) = (A+B)(A+C)
Identidade	A1 = A	A+0 = A
Complemento	A(A') = 0	A+A' = 1
Lei de De Morgan	(AB)' = A'+B'	(A+B)' = A'B'

Essas propriedades são consistentes com nossa compreensão do processamento de portas, bem como com as representações de tabela verdade e diagramas lógicos. Por exemplo, a propriedade comutativa, em termos simples, diz que a ordem dos sinais de entrada não importa, o que é verdade. (Verifique usando as tabelas-verdade de portas individuais.). A propriedade do complemento diz que se colocarmos um sinal e

seu inverso em uma porta AND, temos a garantia de obter 0, mas se colocarmos um sinal e seu inverso em uma porta OR, temos a garantia de obter 1.

? Lei de De Morgan, nomeada em homenagem a Augustus De Morgan

De Morgan, contemporâneo de George Boole, foi o primeiro professor de matemática na Universidade de Londres em 1828, onde continuou a lecionar por 30 anos. Ele escreveu textos elementares sobre aritmética, álgebra, trigonometria e cálculo, bem como artigos sobre a possibilidade de estabelecer um cálculo lógico e o problema fundamental de expressar o pensamento por meio de símbolos. De Morgan não descobriu a lei que leva seu nome, mas é creditado a ele por formulá-la formalmente como é conhecida hoje.[3]

Há um teorema muito famoso – e útil – na álgebra booleana chamado Teorema (ou Lei) de De Morgan. Essa lei afirma que o operador NOT aplicado ao AND de duas variáveis é igual ao NOT aplicado a cada uma das duas variáveis com um OR entre elas. Ou seja, inverter a saída de uma porta AND é equivalente a inverter os sinais individuais primeiro e, em seguida, passá-los por uma porta OR:

$$(AB)' = A' + B'$$

A segunda parte da lei afirma que o operador NOT aplicado ao OR de duas variáveis é igual ao NOT aplicado a cada uma das duas variáveis com um AND entre elas. Expresso em termos de circuito, isso significa que inverter a saída de uma porta OR é equivalente a inverter ambos os sinais de entrada primeiro e depois passá-los por uma porta AND:

$$(A + B)' = A'B'$$

A lei de De Morgan e outras propriedades da álgebra booleana fornecem um mecanismo formal para definir, gerenciar e avaliar projetos de circuitos lógicos.

Somadores

Talvez a operação mais básica que um computador pode realizar seja somar dois números. No nível da lógica digital, essa adição é realizada em binário. O Capítulo 2 discute esse processo em detalhes. Esses tipos de operações de adição são realizados por circuitos especiais chamados, apropriadamente, de somadores.

Assim como a adição em qualquer base, o resultado da adição de dois dígitos binários poderia potencialmente produzir um valor de transporte (vai-um). Lembre-se de que 1 + 1 = 10 na base 2. Um circuito que calcula a soma de dois bits e produz o bit de transporte correto é chamado de meio somador.

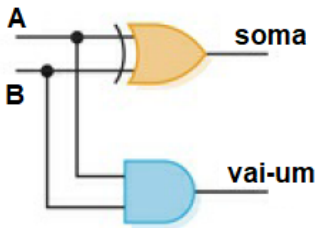
* Somador: Um circuito eletrônico que realiza uma operação de adição em valores binários.

* Meio somador: Um circuito que calcula a soma de dois bits e produz o bit de transporte apropriado.

Vamos considerar todas as possibilidades ao somar dois dígitos binários A e B: Se A e B forem 0, a soma será 0 e o transporte será 0. Se A for 0 e B for 1, a soma será 1 e o transporte será 0. Se A for 1 e B for 0, a soma será 1 e o transporte será 0. Se A e B forem 1, a soma será 0 e o transporte será 1. Isso produz a tabela-verdade ao lado.

A	B	Soma	Vai-um
0	0	0	0
0	1	1	0
1	0	1	0
1	1	0	1

Neste caso, estamos, na verdade, procurando por dois resultados de saída, a soma e o vai-um. Como consequência, nosso circuito tem duas linhas de saída.



Se você comparar as colunas de soma e transporte com a saída das várias portas, verá que a soma corresponde à porta XOR e o vai-um corresponde à porta AND. Assim, o diagrama de circuito ao lado representa um meio somador.

Teste este diagrama atribuindo várias combinações de valores de entrada e determinando os dois valores de saída produzidos. Os resultados seguem as regras da aritmética binária? Eles deveriam. Agora compare seus resultados com a tabela-verdade correspondente. Eles devem corresponder aos resultados ali também.

E quanto à expressão booleana para este circuito? Como o circuito produz dois valores de saída distintos, nós o representamos usando duas expressões booleanas:

$$Soma = A \oplus B \qquad\qquad Vai\text{-}um = A.B$$

Observe que um meio somador não leva em consideração um possível valor de transporte no cálculo (carry-in). Ou seja, um meio somador é adequado para somar dois dígitos únicos, mas não pode ser usado como, pelo menos da forma como está, para calcular a soma de dois valores binários com vários dígitos cada. Um circuito chamado de somador completo leva em consideração o valor de vai-um.

* Somador completo: Um circuito que calcula a soma de dois bits, levando em consideração um bit de transporte de entrada.

Podemos usar dois meio somadores para criar um somador completo. Como? Bem, a entrada para a soma deve ser o vai-um e a soma da adição dos dois valores de entrada. Ou seja, adicionamos a soma do meio somador ao vai-um. Ambas as adições têm um transporte. Poderiam ambos os vai-um serem 1, produzindo mais um transporte? Felizmente, não. Veja a tabela verdade do meio somador. Não há caso em que a soma e o transporte sejam ambos 1.

A FIGURA 4.10 mostra o diagrama lógico e a tabela-verdade para o somador completo. Este circuito possui três entradas: os dois dígitos originais (A e B) e o valor de vai-um. Portanto, a tabela-verdade possui oito linhas. Deixamos a expressão booleana correspondente como exercício.

Diagrama Lógico

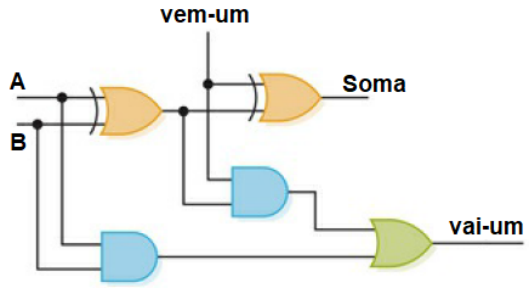


Tabela Verdade

A	B	Vem-um	Soma	Vai-um
0	0	0	0	0
0	0	1	1	0
0	1	0	1	0
0	1	1	0	1
1	0	0	1	0
1	0	1	0	1
1	1	0	0	1
1	1	1	1	1

FIGURA 4.10 Um somador completo.

Para somar dois valores de 8 bits, podemos duplicar um circuito somador completo oito vezes. O vai-um de um valor posicional é usado como vem-um para o próximo valor posicional mais alto. O valor do transporte para a posição do bit mais à direita é assumido como zero, e o transporte da posição do bit mais à esquerda é descartado (potencialmente criando um erro de estouro - overflow).

Existem várias maneiras de aprimorar o projeto desses circuitos somadores, mas não as exploraremos em mais detalhes neste texto.

? Erros

Ao desenvolver seu primeiro programa, por volta de 1949, Maurice Wilkes disse: "Percebi com força total que boa parte do restante da minha vida seria dedicada a encontrar erros em meus próprios programas."[4]

Multiplexadores

Um multiplexador, frequentemente chamado de MUX, é um circuito geral que produz um único sinal de saída. Essa saída é igual a um dos vários sinais de entrada do circuito. O multiplexador seleciona qual sinal de entrada usar como sinal de saída com base no valor representado por mais alguns sinais de entrada, chamados sinais de seleção ou linhas de controle de seleção.

* Multiplexador: Um circuito que usa alguns sinais de controle de entrada para determinar qual das várias linhas de dados de entrada é roteada para sua saída.

Vejamos um exemplo para esclarecer como um multiplexador funciona. A FIGURA 4.11 mostra um diagrama de blocos de um multiplexador. As linhas de controle S0, S1 e S2 determinam quais das outras oito linhas de entrada (D0 a D7) são roteadas para a saída (F).

Os valores das três linhas de controle, tomados em conjunto, são interpretados como um número binário, que determina qual linha de entrada rotear para a saída. Lembre-se do Capítulo 2 que três dígitos binários podem representar oito valores diferentes: 000, 001, 010, 011, 100, 101, 110 e 111. Esses valores, que simplesmente contam em binário de 0 a 7, correspondem aos nossos valores de saída de D0 a D7. Portanto, se S0, S1 e S2 forem todos 0, a linha de entrada D0 seria a saída do multiplexador. Se S0 for 1, S1 for 0 e S2 for 1, então D5 seria a saída do MUX.

A tabela verdade a seguir mostra como as linhas de controle de entrada determinam a saída para este multiplexador. O diagrama de blocos da FIGURA 4.11 esconde um circuito bastante complexo que executa a lógica de um multiplexador. Tal circuito poderia ser mostrado usando oito portas AND de três entradas e uma porta OR de oito entradas. Não entraremos em detalhes sobre este circuito neste livro.

.

Diagrama de Blocos



Tabela Verdade

S0	S1	S2	F
0	0	0	D1
0	0	1	D2
0	1	0	D3
0	1	1	D4
1	0	0	D5
1	0	1	D6
1	1	0	D7
1	1	1	D8

FIGURA 4.11 Diagrama de blocos de um multiplexador com três linhas de controle de seleção.

Um multiplexador pode ser projetado com vários números de linhas de entrada e linhas de controle correspondentes. Em geral, os valores binários em n linhas de controle de entrada são usados para determinar quais das outras 2^n linhas de dados são selecionadas para saída.

Um circuito chamado demultiplexador (DEMUX) realiza a operação oposta. Ou seja, ele recebe uma única entrada e a encaminha para uma das 2^n saídas, dependendo dos valores das n linhas de controle.

4.5 Circuitos como Memória

Circuitos digitais desempenham outro papel importante: eles podem armazenar informações. Esses circuitos formam um circuito sequencial, pois a saída do circuito também serve como entrada para o circuito. Ou seja, o estado atual do circuito é usado em parte para determinar o próximo estado.

Muitos tipos de circuitos de memória já foram projetados. Examinamos apenas um tipo neste livro: o latch S-R. Um latch S-R armazena um único dígito binário (1 ou 0). Um circuito de latch S-R poderia ser projetado usando uma variedade de portas. Um desses circuitos, usando portas NAND, é ilustrado na FIGURA 4.12.

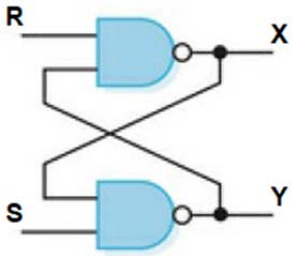


FIGURA 4.12 Um latch S-R.

O projeto deste circuito garante que as duas saídas X e Y sejam sempre complementares uma da outra. Ou seja, quando X é 0, Y é 1 e vice-versa. O valor de X em qualquer momento é considerado o estado atual do circuito. Portanto, se X for 1, o circuito está armazenando 1; se X for 0, o circuito está armazenando 0.

Lembre-se de que uma porta NAND produz uma saída 1, a menos que ambos os seus valores de entrada sejam 1. Cada porta neste circuito tem uma entrada externa (S ou R) e uma entrada proveniente da saída da outra porta. Suponha que o estado atual do circuito esteja armazenando 1 (ou seja, X é 1), e suponha que S e R sejam 1. Então Y permanece 0 e X permanece 1. Agora, suponha que o circuito esteja atualmente armazenando 0 (X é 0) e que R e S sejam novamente 1. Então Y permanece 1 e X permanece 0. Independentemente do valor que esteja sendo armazenado no momento, se ambos os valores de entrada S e R forem 1, o circuito mantém seu estado atual.

Esta explicação demonstra que o latch S-R mantém seu valor enquanto S e R forem 1. Mas como um valor é armazenado em primeiro lugar? Definimos o latch S-R como 1, definindo S momentaneamente como 0, mantendo R em 1. Se S for 0, X se torna 1. Enquanto S for retornado a 1 imediatamente, o latch S-R permanece no estado 1. Definimos o latch como 0, definindo R momentaneamente como 0, mantendo S em 1. Se R for 0, Y se torna 0 e, portanto, X se torna 0. Enquanto R for imediatamente redefinido para 1, o estado do circuito permanece 0.

Controlando cuidadosamente os valores de S e R, o circuito pode ser configurado para armazenar qualquer um dos valores. Ao escalar essa ideia para circuitos maiores, podemos projetar dispositivos de memória com capacidades maiores.

4.6 Circuitos Integrados

Um circuito integrado (também chamado de chip) é uma peça de silício na qual múltiplas portas foram embutidas. Essas peças de silício são montadas em um encapsulamento de plástico ou cerâmica com pinos ao longo das bordas que podem ser soldados em placas de circuito ou inseridos em soquetes apropriados. Cada pino se conecta à entrada ou saída de uma porta, ou à alimentação ou ao aterramento.

* Circuito integrado (chip) Uma peça de silício na qual múltiplas portas foram embutidas.

Os circuitos integrados (CI) são classificados pelo número de portas que contêm. Essas classificações também refletem o desenvolvimento histórico da tecnologia de CI:

Abreviação	Nome	Número de portas
SSI	Integração em pequena escala	1 a 10
MSI	Integração em média escala	10 a 100
LSI	Integração em grande escala (LSI)	100 a 100.000
VLSI	Integração em grande escala (VLSI)	acima 100.000

Um chip SSI possui algumas portas independentes, como a mostrada na FIGURA 4.13. Este chip possui 14 pinos: oito para entradas para portas, quatro para saída das portas, um para aterramento e um para alimentação. Chips semelhantes podem ser fabricados com portas diferentes.

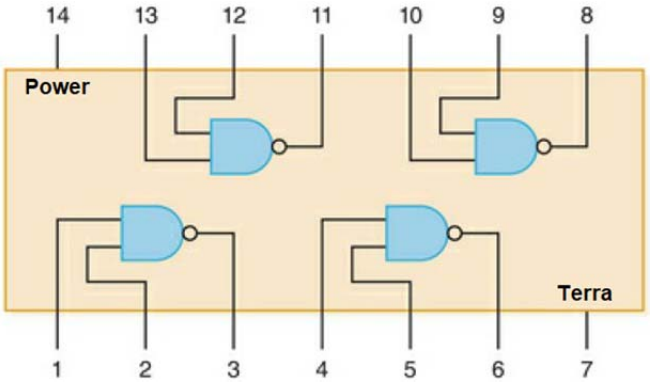


FIGURA 4.13 Um chip SSI contendo portas NAND independentes.

Como um chip pode ter mais de 100.000 portas? Isso implicaria na necessidade de 300.000 pinos! A questão é que as portas em um chip VLSI não são independentes, pois estão em integração em pequena escala. Os chips VLSI incorporam circuitos com uma alta relação porta-pino. Ou seja, muitas portas são combinadas para criar circuitos complexos que requerem apenas alguns valores de entrada e saída. Multiplexadores são um exemplo desse tipo de circuito.

? O que é ética da computação?

Cuidado – o termo ética da computação é ambíguo. O uso do termo na décima vertente dos currículos de ciência da computação refere-se a um código de ética que os profissionais da computação podem aplicar em sua própria profissão. A ética da computação também se refere às determinações feitas por filósofos contemporâneos em casos que envolvem computadores ou redes de computadores.

4.7 Chips de CPU

O circuito integrado mais importante em qualquer computador é a unidade central de processamento (CPU). O processamento de uma CPU será discutido no próximo capítulo, mas é importante reconhecer neste ponto que a CPU é, em certo sentido, meramente um circuito avançado com linhas de entrada e saída.

Cada chip de CPU contém um grande número de pinos através dos quais ocorre essencialmente toda a comunicação em um sistema de computador. Essa comunicação conecta a CPU à memória e aos dispositivos de E/S, que são, em níveis fundamentais, circuitos avançados.

A explicação do processamento da CPU e sua interação com outros dispositivos nos leva a outro nível de processamento computacional, às vezes chamado de arquitetura de componentes. Embora ainda seja focada principalmente em hardware, a arquitetura de componentes de computador aplica o princípio da abstração

mais uma vez, permitindo-nos ignorar temporariamente os detalhes das portas e circuitos discutidos neste capítulo e nos aproximando cada vez mais de uma compreensão completa do processamento computacional.

RESUMO

Neste capítulo, discutimos como um computador opera em seu nível mais baixo, controlando o fluxo de eletricidade. Como estamos lidando com computadores digitais que utilizam informações binárias, nos preocupamos com apenas duas faixas de tensão, que interpretamos como binárias 0 ou 1. O fluxo de eletricidade é guiado por dispositivos eletrônicos chamados portas, que realizam operações lógicas básicas como NOT, AND e OR. Uma porta é criada usando um ou mais transistores, uma invenção que revolucionou a computação.

As portas podem ser combinadas em circuitos, nos quais a saída de uma porta serve como entrada para outra porta. Projetando esses circuitos cuidadosamente, podemos criar dispositivos que realizam tarefas mais complexas, como somar, multiplexar e armazenar dados. Conjuntos de portas, ou circuitos completos, são frequentemente incorporados em um único circuito integrado, ou chip, o que leva ao conceito de unidade central de processamento (CPU).

EXERCÍCIOS

Para os Exercícios 1 a 17, marque as respostas como verdadeiras ou falsas, da seguinte forma:

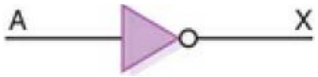
A. Verdadeiro B. Falso

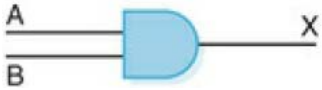
1. Diagramas lógicos e tabelas-verdade são igualmente poderosos para expressar o processamento de portas e circuitos.
2. Expressões booleanas são mais poderosas do que diagramas lógicos para expressar o processamento de portas e circuitos.
3. Uma porta NOT aceita duas entradas.
4. O valor de saída de uma porta AND é 1 quando ambas as entradas são 1.
5. As portas AND e OR produzem resultados opostos para a mesma entrada.
6. O valor de saída de uma porta OR é 1 quando ambas as entradas são 1.
7. A saída de uma porta OR é 0 quando uma entrada é 0 e a outra é 1.
8. O valor de saída de uma porta XOR é 0, a menos que ambas as entradas sejam 1.
9. A porta NOR produz resultados opostos aos da porta XOR.
10. Uma porta pode ser projetada para aceitar mais de duas entradas.
11. Um transistor é feito de material semicondutor.
12. Inverter a saída de uma porta AND equivale a inverter os sinais individuais primeiro e, em seguida, passá-los por uma porta OR.
13. A soma de dois dígitos binários (ignorando o carry) é expressa por uma porta AND.
14. Um somador completo leva em consideração o valor do carry-in.
15. Um multiplexador soma todos os bits em suas linhas de entrada para produzir sua saída.
16. Os circuitos integrados são classificados pelo número de portas que contêm.
17. Uma CPU é um circuito integrado.

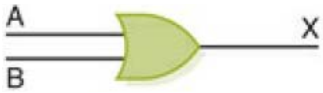
Para os Exercícios 18 a 29, combine a porta com a descrição da operação ou o diagrama.

A. AND B. NAND C. XOR D. OR E. NOR F. NOT

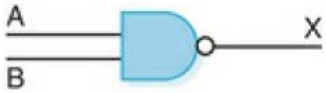
18. Inverte sua entrada.
19. Produz 1 somente se todas as suas entradas forem 1 e 0 caso contrário.
20. Produz 0 somente se todas as suas entradas forem 0 e 1 caso contrário.
21. Produz 0 somente se suas entradas forem iguais e 1 caso contrário.

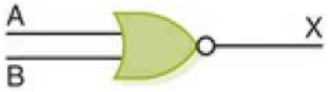
22. Produz 0 se todas as suas entradas forem 1 e 1 caso contrário.
23. Produz 1 se todas as suas entradas forem 0 e 0 caso contrário.
24. 

25. 

26. 

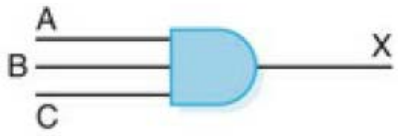
27. 

28. 

29. 

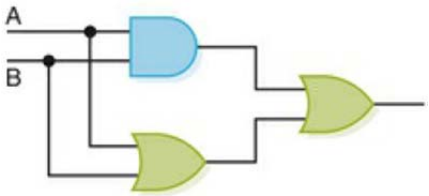
Os exercícios 30 a 58 são questões de resposta curta ou de projeto.

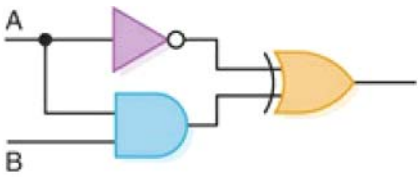
30. Como o nível de tensão é usado para distinguir entre dígitos binários?
31. Distinguir entre uma porta e um circuito.
32. Quais são os três métodos de notação para descrever o comportamento de portas e circuitos?
33. Caracterize as notações solicitadas no Exercício 32.
34. Quantos sinais de entrada uma porta pode receber e quantos sinais de saída uma porta pode produzir?
35. Nomeie seis tipos de portas.
36. Dê as três representações de uma porta NOT e diga em palavras o que NOT significa.
37. Dê as três representações de uma porta AND e diga em palavras o que AND significa.
38. Dê as três representações de uma porta OR e diga em palavras o que OR significa.
39. Dê as três representações de uma porta XOR e diga em palavras o que XOR significa.
40. Apresente as três representações de uma porta NAND e diga em palavras o que NAND significa.
41. Apresente as três representações de uma porta NOR e diga em palavras o que NOR significa.
42. Compare e contraste a porta AND e a porta NAND.
43. Considere a figura ao lado. Apresente a expressão booleana para uma porta AND de três entradas e, em seguida, mostre seu comportamento com uma tabela-verdade.

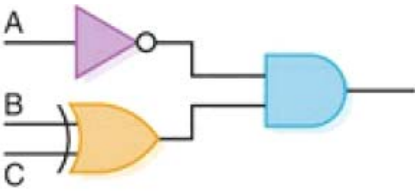


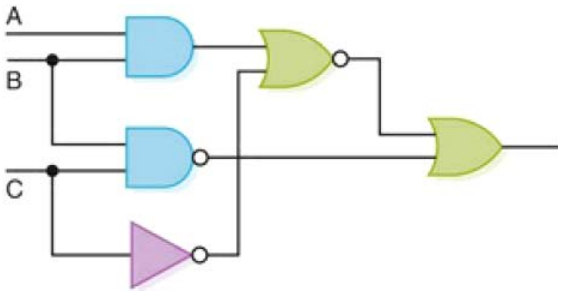
44. Dê a expressão booleana para uma porta OR de três entradas e, em seguida, mostre seu comportamento com uma tabela-verdade.
45. O que é usado em uma porta para estabelecer como os valores de entrada se relacionam com o valor de saída?
46. Como um transistor se comporta?
47. Do que um transistor é feito?
48. O que acontece quando um sinal elétrico é aterrado?
49. Quais são os três terminais de um transistor e como eles operam?
50. Quantos transistores são necessários para cada uma dessas portas?
    a) NOT b) AND c) NOR d) OR e) XOR
51. Desenhe um diagrama de transistor para uma porta E. Explique o processamento.
52. Desenhe um diagrama de transistor para uma porta OU. Explique o processamento.
53. Como as portas podem ser combinadas em circuitos?
54. Quais são as duas categorias gerais de circuitos e como elas diferem?
55. Desenhe um diagrama de circuito correspondente à seguinte expressão booleana: (A + B)(B + C)
56. Desenhe um diagrama de circuito correspondente à seguinte expressão booleana: (AB + C)D
57. Desenhe um diagrama de circuito correspondente à seguinte expressão booleana: A'B + (B + C)'
58. Desenhe um diagrama de circuito correspondente à seguinte expressão booleana: (AB)' + (CD)'

Os exercícios 59 a 62 são questões de resposta curta ou de projeto. Mostre o comportamento dos circuitos com uma tabela-verdade.

59. 

60. 

61. 

62. 

Os exercícios 63 a 73 são questões de resposta curta ou de projeto.

63. O que é equivalência de circuitos?
64. Cite seis propriedades da álgebra booleana e explique o que cada uma significa.
65. Diferencie um meio somador de um somador completo.
66. Qual é a expressão booleana para um somador completo?
67. O que é um multiplexador?
68. Responda as perguntas abaixo:
    a) Quais são os tipos de circuitos usados para memória?
    b) Quantos dígitos um latch S-R armazena?
    c) O projeto de um latch S-R mostrado na Figura 4.12 garante o que sobre as saídas X e Y?
69. O que é um circuito integrado ou chip?
70. Defina as abreviações SSI, MSI, LSI e VLSI.
71. No chip mostrado na Figura 4.13, para que servem os pinos?
72. Desenhe um circuito usando dois somadores completos que somam dois valores binários de dois bits. Mostre sua tabela verdade correspondente.
73. Como a operação XOR pode ser expressa usando outros operadores?

QUESTÕES PARA REFLEXÃO

1. Ao longo deste capítulo, usamos expressões booleanas, tabelas-verdade e diagramas lógicos para descrever o mesmo comportamento. A relação entre esses métodos de notação está clara para você? Qual você considera o mais intuitivo? Qual você considera o menos intuitivo?
2. Muitas situações podem ser descritas pelas ideias deste capítulo – por exemplo, o funcionamento de um único interruptor de luz ou de uma luz com dois interruptores. Você consegue pensar em outras ocorrências cotidianas que podem ser descritas pelos métodos de notação apresentados neste capítulo?
3. Como os dois conjuntos de códigos de ética diferem? Como eles são semelhantes?
4. Você já enviou um e-mail para alguém e se arrependeu imediatamente? Você acha que diria algo por e-mail que nunca diria pessoalmente? Considere a seguinte premissa: "O e-mail diminuiu a civilidade do discurso pessoal". Você concorda ou discorda?
5. Se uma pessoa envia um e-mail de um computador escolar ou comercial, essa mensagem deve ser considerada privada? A instituição ou pessoa proprietária do computador de onde o e-mail é enviado tem o direito de inspecionar a mensagem?